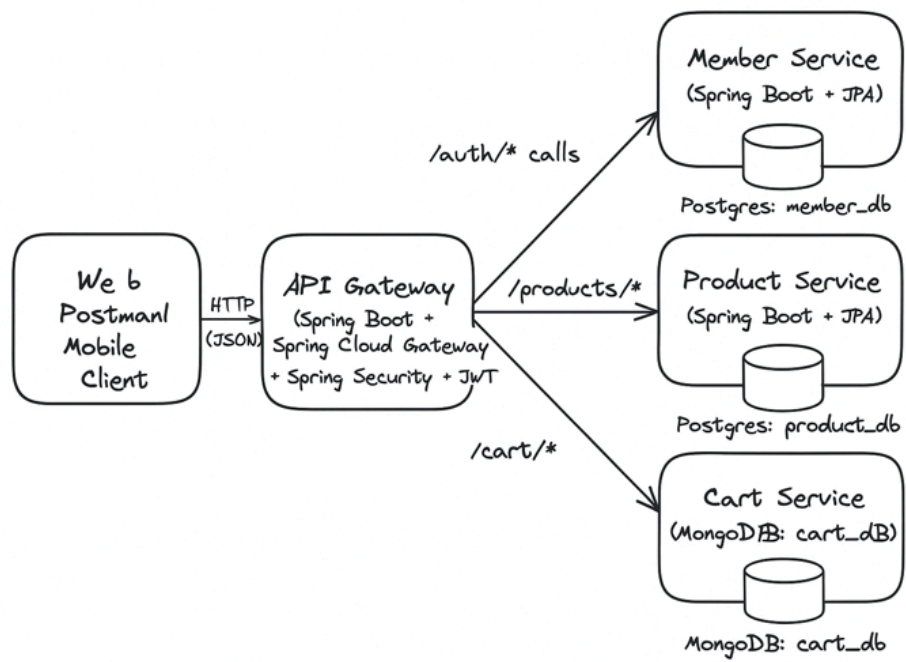
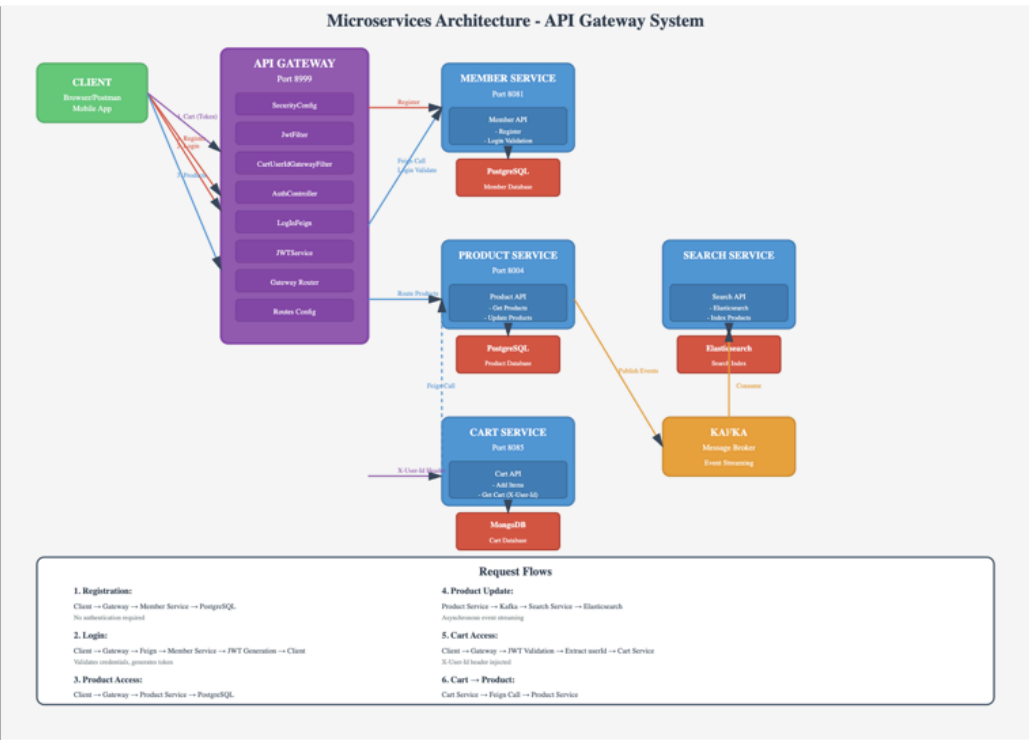


Online Market Place

Flow Diagram



Architectural Diagram



1. Member Service – PostgreSQL

1.1 Overview

- **Database:** member_db
- **Main table:** members
- Purpose: store registered customers with secure (hashed) password.

1.2 members table

Table name: members

Field	Java Type	Column Name	Constraints
id	Long	id	PRIMARY KEY, AUTO_INCREMENT
email	String	email	NOT NULL, UNIQUE, VARCHAR(255)
passwordHash	String	password_hash	NOT NULL, VARCHAR(255)
name	String	name	NOT NULL, VARCHAR(255)
phoneNumber	String	phone_number	NOT NULL, UNIQUE, VARCHAR(255)
status	String	status	NOT NULL, DEFAULT 'ACTIVE', VARCHAR(50)
createdAt	Date	created_at	NOT NULL, TIMESTAMP, AUTO
updatedAt	Date	updated_at	NOT NULL, TIMESTAMP, AUTO

2. Product Service – PostgreSQL

2.1 Overview

- **Database:** product_db
- **Main table:** products
- Purpose: store all product catalog information and support wildcard search + pagination.

2.2 products table

Table name: products

Column	Type	Constraints / Indexes	Description
id	BIGINT	PRIMARY KEY	Unique product identifier
store_id	INT	NOT NULL , default 10001	Default Store Id : 10001
sku_id	VARCHAR(100)	NOT NULL , UNIQUE	Unique Product code
name	VARCHAR(255)	NOT NULL	Product name (used in search)
description	TEXT	NULL	Product description
category	VARCHAR(100)	NULL	Category name (e.g. "Laptops")
brand	VARCHAR(100)	NULL	Brand name
price	BIGINT	NOT NULL	Price of the Product
item_code	BIGINT	NOT NULL	Unique item identifier

is_active	BOOLEAN	NOT NULL , default TRUE	Whether product is ACTIVE, INACTIVE
length	BIGINT	NOT NULL	Length of the Product
height	BIGINT	NOT NULL	Height of the Product
width	BIGINT	NOT NULL	Width of the Product
weight	BIGINT	NOT NULL	Weight of the Product
dangerous_level	INT	NOT NULL	Dangerous Level of the Product
created_at	TIMESTAMP	NOT NULL	Record creation timestamp
updated_at	TIMESTAMP	NOT NULL	Last update timestamp

Indexes:

- idx_products_sku on sku_id
- idx_products_name on name
- idx_products_category on category
- idx_products_brand on brand

3. Cart Service – MongoDB

3.1 Overview

- **Database:** cart_db
- **Collection:** carts
- Purpose: maintain each customer's shopping cart as a single document.

3.2 carts collection document structure

Collection name: carts

One **cart document per user**.

Document example

```
1 {
2   "_id": "cart_12345",
3   "userId": 12345,
4   "items": [
5     {
6       "skuId": "MTA-8935-56768",
7       "productId": 1001,
8       "quantity": 2,
9       "price": 199900,
10      "addedAt": "2025-11-29T10:00:00Z"
11    },
12    {
13      "skuId": "MTA-8935-56769",
14      "productId": 1002,
15      "quantity": 1,
16      "price": 499900,
17      "addedAt": "2025-11-29T10:05:00Z"
18    }
19  ],
20  "totalPrice": 699800,
21  "totalQuantity": 3,
22  "updatedAt": "2025-11-29T10:05:00Z"
23 }
```

Field descriptions:

Field	Type	Description
<code>_id</code>	String	Unique cart identifier (not ObjectId)
<code>userId</code>	Long	User ID who owns this cart
<code>items</code>	Array	List of cart items
<code>items[].skuId</code>	String	Product SKU identifier
<code>items[].productId</code>	Long	Product ID
<code>items[].quantity</code>	Integer	Quantity of this item in cart
<code>items[].price</code>	Long	Price of the item (in smallest currency unit,

		e.g., cents)
<code>items[].addedAt</code>	Date	Timestamp when item was added to cart
<code>totalPrice</code>	Long	Sum of all item prices (price × quantity)
<code>totalQuantity</code>	Integer	Total number of items in cart
<code>updatedAt</code>	Date	Last update timestamp of the cart

4. Redis (Product)

Cache Name	Purpose	Key Pattern
<code>product</code>	Individual product cache	<code>product:{skuId}</code>
<code>products</code>	Search results cache	<code>products:search:{term}:page:{page}:size:{size}</code>

5. Search - Elasticsearch Index Schema

Field Name	Java Type	Elasticsearch Field Type	Index Name	Analyzer	Constraints / Notes
<code>skuId</code>	<code>String</code>	<code>_id</code> (Document ID)	<code>products</code>	N/A	PRIMARY KEY - Unique identifier, used as Elasticsearch

					document _id . Required, non-null.
storeId	Integer	integer	products	N/A	Store identifier. Can be null.
name	String	text	products	standard	Product name. Searchable (full- text). Normalized (lowercase , spaces removed) for space- handling. Can be null.
description	String	text	products	standard	Product description. Searchable (full- text). Normalized (lowercase , spaces removed) for space- handling.

					Can be null.
category	String	keyword	products	N/A	Product category. Searchable (exact match). Normalized (lowercase, spaces removed) for case-insensitive and space-handling search. Can be null.
brand	String	keyword	products	N/A	Product brand. Searchable (exact match). Normalized (lowercase, spaces removed) for case-insensitive and space-handling search.

					Can be null.
price	Long	long	products	N/A	Product price in smallest currency unit. Can be null.
itemCode	Long	long	products	N/A	Item code identifier. Can be null.
isActive	Boolean	boolean	products	N/A	Product active status. Can be null.
length	Long	long	products	N/A	Product length. Can be null.
height	Long	long	products	N/A	Product height. Can be null.
width	Long	long	products	N/A	Product width. Can be null.
weight	Long	long	products	N/A	Product weight. Can be null.

<code>dangerousLevel</code>	<code>Integer</code>	<code>integer</code>	<code>products</code>	N/A	Dangerous goods level. Can be null.
<code>createdAt</code>	<code>LocalDate</code>	<code>date</code>	<code>products</code>	N/A	Creation timestamp . Format: <code>yyyy-MM-dd</code> . Can be null.
<code>updatedAt</code>	<code>LocalDate</code>	<code>date</code>	<code>products</code>	N/A	Last update timestamp . Format: <code>yyyy-MM-dd</code> . Can be null.

1. API Gateway – Auth APIs (public)

1.1 POST /api/member/login

Description: Authenticates a user with email and password. The gateway validates credentials via Member Service using Feign client, and generates a JWT token upon successful authentication.

Method: `POST`

Path: `/api/member/login`

Authentication: Not required (public endpoint)

Request

Headers:

```
1 Content-Type: application/json
2 Accept: application/json
```

Request Body:

```
1 {
2   "email": "user@example.com",
3   "password": "userPassword123"
4 }
```

Request Body Schema:

Field	Type	Required	Validation	Description
email	String	Yes	Valid email format	User's email address
password	String	Yes	Not blank	User's password

Response

Success Response (200 OK):

```
1 {
2   "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiIxMDk4MCI6Im1hdCI6MTc2NDcyOTIzMiwiZmxwIjoxNzY0ODM3MjMyfQ.XcJs96in8vE4fDoKJ9uiKtB5jRKBLvyi2uaNKGba1xQ",
3   "message": "Login successful"
4 }
```

Response Schema:

Field	Type	Description
token	String	JWT token (valid for 1 hour). Include in Authorization: Bearer <token> header for protected endpoints
message	String	Success message

Error Responses:

401 Unauthorized — Invalid credentials:

```
1 "Invalid credentials or member not found"
```

500 Internal Server Error — Server error:

```
1 | "Login failed: <error message>"
```

2. Member Service APIs (Internal)

These are **not exposed directly to clients**; Gateway calls them.

2.1. POST /api/member/register

Description: Register a new member/customer with hashed password.

Endpoint: POST /api/member/register

Request Headers:

```
1 | Content-Type: application/json
```

Request Body:

```
1 | {
2 |   "email": "user@example.com",
3 |   "password": "StrongPassword123",
4 |   "name": "John Doe",
5 |   "phoneNumber": "1234567890",
6 |   "address": "123 Main Street"
7 | }
```

Success Response (201 Created):

```
1 | {
2 |   "success": true,
3 |   "errorMessage": null,
4 |   "errorCode": null,
5 |   "status": 201,
6 |   "statusText": "CREATED",
7 |   "timestamp": "2025-12-05T10:30:00.123Z",
8 |   "data": {
9 |     "id": 12345,
10 |    "email": "user@example.com",
11 |    "name": "John Doe",
12 |    "message": "Registered the user successfully"
13 |   }
14 | }
```

Error Responses:

400 Bad Request - Validation Error:

```
1 | {
2 |   "timestamp": "2025-12-05T10:30:00.123Z",
3 |   "status": 400,
4 |   "error": "Validation Error",
5 |   "message": "Invalid email format. Email must contain a valid domain w
ith extension"
6 | }
```

409 Conflict - Duplicate Email:

```
1 {
2   "timestamp": "2025-12-05T10:30:00.123Z",
3   "status": 409,
4   "error": "Business Rule Violation",
5   "message": "Email address is already registered. Please use a different email or try logging in."
6 }
```

409 Conflict - Duplicate Phone Number:

```
1 {
2   "timestamp": "2025-12-05T10:30:00.123Z",
3   "status": 409,
4   "error": "Business Rule Violation",
5   "message": "Phone number is already registered. Please use a different phone number."
6 }
```

500 Internal Server Error:

```
1 {
2   "timestamp": "2025-12-05T10:30:00.123Z",
3   "status": 500,
4   "error": "Internal Server Error",
5   "message": "An unexpected error occurred. Please try again later."
6 }
```

2.2 POST /api/member/login

Description: Validate user credentials and authenticate login.

Endpoint: POST /api/member/login

Request Headers:

```
1 Content-Type: application/json
```

Request Body:

```
1 {
2   "email": "user@example.com",
3   "password": "StrongPassword123"
4 }
```

Success Response (200 OK):

```
1 {
2   "success": true,
3   "errorMessage": null,
4   "errorCode": null,
5   "status": 200,
6   "statusText": "OK",
7   "timestamp": "2025-12-05T10:30:00.123Z",
8   "data": {
9     "isMember": true,
10    "userId": 12345
11  }
12 }
```

Error Responses:

400 Bad Request - User Not Found:

```
1 {  
2   "timestamp": "2025-12-05T10:30:00.123Z",  
3   "status": 400,  
4   "error": "Validation Error",  
5   "message": "User Not Found"  
6 }
```

400 Bad Request - Incorrect Password:

```
1 {  
2   "timestamp": "2025-12-05T10:30:00.123Z",  
3   "status": 400,  
4   "error": "Validation Error",  
5   "message": "Incorrect Password"  
6 }
```

400 Bad Request - Invalid Email Format:

```
1 {  
2   "timestamp": "2025-12-05T10:30:00.123Z",  
3   "status": 400,  
4   "error": "Validation Error",  
5   "message": "Invalid email format. Email must contain a valid domain w  
ith extension (e.g., .com, .org)"  
6 }
```

500 Internal Server Error:

```
1 {  
2   "timestamp": "2025-12-05T10:30:00.123Z",  
3   "status": 500,  
4   "error": "Internal Server Error",  
5   "message": "An unexpected error occurred. Please try again later."  
6 }
```

3. Product APIs (public via Gateway)

Base: `/api/products`

3.1 GET `/api/products`

Description: List products with pagination

Query Parameters:

- `page` (optional, default: 0) - Page number (0-indexed)
- `size` (optional, default: 10) - Number of items per page

Example Request:

```
1 GET /api/products?page=0&size=20
```

Response: 200 OK

```
1 {
2   "success": true,
3   "errorMessage": null,
4   "errorCode": null,
5   "status": 200,
6   "statusText": "OK",
7   "timestamp": "2025-12-07T10:30:02.260641Z",
8   "data": {
9     "content": [
10      {
11        "skuId": "LAP-001",
12        "storeId": 10001,
13        "name": "Gaming Laptop X",
14        "description": "High performance gaming laptop",
15        "category": "Laptops",
16        "brand": "BrandX",
17        "price": 799900,
18        "itemCode": 123456789,
19        "isActive": true,
20        "length": 35,
21        "height": 2,
22        "width": 25,
23        "weight": 2000,
24        "dangerousLevel": 0,
25        "createdAt": "2025-11-01T10:00:00",
26        "updatedAt": "2025-11-01T10:00:00"
27      }
28    ],
29    "page": 0,
30    "size": 20,
31    "totalElements": 50000,
32    "totalPages": 2500
33  }
34 }
```

3.2 GET /api/products/searchByTerm

Description: Wildcard search on product name/description + pagination. Returns empty list if no matches found.

Query Parameters:

- `searchTerm` (required) - Search term for product name/description
- `page` (optional, default: 0) - Page number (0-indexed)
- `size` (optional, default: 10) - Number of items per page

Example Request:

```
1 GET /api/products/searchByTerm?searchTerm=laptop&page=0&size=10
```

Response: 200 OK

```
1 {
2   "success": true,
3   "errorMessage": null,
```

```

4   "errorCode": null,
5   "status": 200,
6   "statusText": "OK",
7   "timestamp": "2025-12-07T10:30:02.260641Z",
8   "data": {
9     "content": [
10      {
11        "skuId": "LAP-001",
12        "storeId": 10001,
13        "name": "Gaming Laptop X",
14        "description": "High performance gaming laptop",
15        "category": "Laptops",
16        "brand": "BrandX",
17        "price": 799900,
18        "itemCode": 123456789,
19        "isActive": true,
20        "length": 35,
21        "height": 2,
22        "width": 25,
23        "weight": 2000,
24        "dangerousLevel": 0,
25        "createdAt": "2025-11-01T10:00:00",
26        "updatedAt": "2025-11-01T10:00:00"
27      }
28    ],
29    "page": 0,
30    "size": 10,
31    "totalElements": 1234,
32    "totalPages": 124
33  }
34 }

```

Error Response: 400 Bad Request (if searchTerm is empty)

```

1  {
2    "error": "Validation Error",
3    "message": "Search term is required and cannot be empty",
4    "timestamp": "2025-12-07T10:30:02.260641",
5    "status": 400
6  }

```

3.3 GET /api/products/{skuId}

Description: Get product by SKU ID

Path Parameters:

- skuId (required, String) - Unique product SKU identifier

Example Request:

```

1 GET /api/products/LAP-001

```

Response: 200 OK

```

1  {
2    "success": true,
3    "errorMessage": null,
4    "errorCode": null,
5    "status": 200,

```



```
6  "statusText": "OK",
7  "timestamp": "2025-12-07T10:30:02.260641Z",
8  "data": {
9    "skuId": "LAP-001",
10   "storeId": 10001,
11   "name": "Gaming Laptop X",
12   "description": "High performance gaming laptop with 16GB RAM, 1TB
SSD",
13   "category": "Laptops",
14   "brand": "BrandX",
15   "price": 799900,
16   "itemCode": 123456789,
17   "isActive": true,
18   "length": 35,
19   "height": 2,
20   "width": 25,
21   "weight": 2000,
22   "dangerousLevel": 0,
23   "createdAt": "2025-11-01T10:00:00",
24   "updatedAt": "2025-11-01T10:00:00"
25 }
26 }
```

Error Response: 404 Not Found

```
1 {
2   "error": "Resource Not Found",
3   "message": "Product not found with id: LAP-001",
4   "timestamp": "2025-12-07T10:30:02.260641",
5   "status": 404
6 }
```

3.4 POST /api/products/getSkusById

Description: Get multiple products by their SKU IDs

Request Body:

```
1 [
2   "LAP-001",
3   "LAP-002",
4   "PHN-001"
5 ]
```

Example Request:

```
1 POST /api/products/getSkusById
2 Content-Type: application/json
3
4 ["LAP-001", "LAP-002", "PHN-001"]
```

Response: 200 OK

```
1 {
2   "success": true,
3   "errorMessage": null,
4   "errorCode": null,
5   "status": 200,
6   "statusText": "OK",
7   "timestamp": "2025-12-07T10:30:02.260641Z",
```

```

8  "data": [
9    {
10     "skuId": "LAP-001",
11     "storeId": 10001,
12     "name": "Gaming Laptop X",
13     "description": "High performance gaming laptop",
14     "category": "Laptops",
15     "brand": "BrandX",
16     "price": 799900,
17     "itemCode": 123456789,
18     "isActive": true,
19     "length": 35,
20     "height": 2,
21     "width": 25,
22     "weight": 2000,
23     "dangerousLevel": 0,
24     "createdAt": "2025-11-01T10:00:00",
25     "updatedAt": "2025-11-01T10:00:00"
26   },
27   {
28     "skuId": "LAP-002",
29     "storeId": 10001,
30     "name": "Business Laptop Y",
31     "description": "Professional business laptop",
32     "category": "Laptops",
33     "brand": "BrandY",
34     "price": 599900,
35     "itemCode": 123456790,
36     "isActive": true,
37     "length": 33,
38     "height": 2,
39     "width": 23,
40     "weight": 1800,
41     "dangerousLevel": 0,
42     "createdAt": "2025-11-01T10:00:00",
43     "updatedAt": "2025-11-01T10:00:00"
44   }
45 ]
46 }

```

Error Response: 400 Bad Request (if SKU IDs list is null or empty)

```

1  {
2    "error": "Validation Error",
3    "message": "SKU IDs list cannot be null or empty",
4    "timestamp": "2025-12-07T10:30:02.260641",
5    "status": 400
6  }

```

Error Response: 404 Not Found (if no products found)

```

1  {
2    "error": "Resource Not Found",
3    "message": "No products found for the given SKU IDs",
4    "timestamp": "2025-12-07T10:30:02.260641",
5    "status": 404
6  }

```

3.5 GET /api/products/isPresent

Description: Check if a product exists by SKU ID

Query Parameters:

- `skuId` (required, String) - Unique product SKU identifier

Example Request:

```
1 GET /api/products/isPresent?skuId=LAP-001
```

Response: 200 OK

```
1 {
2   "success": true,
3   "errorMessage": null,
4   "errorCode": null,
5   "status": 200,
6   "statusText": "OK",
7   "timestamp": "2025-12-07T10:30:02.260641Z",
8   "data": true
9 }
```

3.6 POST /api/products/addProduct

Description: Add a new product to the catalog

Request Body:

```
1 {
2   "skuId": "LAP-001",
3   "name": "Gaming Laptop X",
4   "description": "High performance gaming laptop",
5   "category": "Laptops",
6   "brand": "BrandX",
7   "price": 799900,
8   "itemCode": 123456789,
9   "length": 35,
10  "height": 2,
11  "width": 25,
12  "weight": 2000,
13  "dangerousLevel": 0
14 }
```

Example Request:

```
1 POST /api/products/addProduct
2 Content-Type: application/json
3
4 {
5   "skuId": "LAP-001",
6   "name": "Gaming Laptop X",
7   "description": "High performance gaming laptop",
8   "category": "Laptops",
9   "brand": "BrandX",
10  "price": 799900,
11  "itemCode": 123456789,
12  "length": 35,
13  "height": 2,
14  "width": 25,
15  "weight": 2000,
16  "dangerousLevel": 0
}
```

```
17 }
```

Response: 201 Created

```
1 {
2   "success": true,
3   "errorMessage": null,
4   "errorCode": null,
5   "status": 201,
6   "statusText": "Created",
7   "timestamp": "2025-12-07T10:30:02.260641Z",
8   "data": {
9     "skuId": "LAP-001",
10    "storeId": 10001,
11    "name": "Gaming Laptop X",
12    "description": "High performance gaming laptop",
13    "category": "Laptops",
14    "brand": "BrandX",
15    "price": 799900,
16    "itemCode": 123456789,
17    "isActive": true,
18    "length": 35,
19    "height": 2,
20    "width": 25,
21    "weight": 2000,
22    "dangerousLevel": 0,
23    "createdAt": "2025-12-07T10:30:02.260641",
24    "updatedAt": "2025-12-07T10:30:02.260641"
25  }
26 }
```

Error Response: 400 Bad Request (validation errors)

```
1 {
2   "error": "Validation Error",
3   "message": "skuId is required",
4   "timestamp": "2025-12-07T10:30:02.260641",
5   "status": 400
6 }
```

Error Response: 409 Conflict (duplicate SKU ID)

```
1 {
2   "error": "Business Rule Violation",
3   "message": "A product with SKU ID 'LAP-001' already exists. Please use a different SKU ID.",
4   "timestamp": "2025-12-07T10:30:02.260641",
5   "status": 409
6 }
```

3.7 PUT /api/products/update/{skuId}

Description: Update an existing product

Path Parameters:

- `skuId` (required, String) - Unique product SKU identifier to update

Request Body:

```

1  {
2    "skuId": "LAP-001",
3    "name": "Gaming Laptop X Pro",
4    "description": "Updated high performance gaming laptop",
5    "category": "Laptops",
6    "brand": "BrandX",
7    "price": 899900,
8    "itemCode": 123456789,
9    "length": 35,
10   "height": 2,
11   "width": 25,
12   "weight": 2000,
13   "dangerousLevel": 0
14 }

```

Example Request:

```

1  PUT /api/products/update/LAP-001
2  Content-Type: application/json
3
4  {
5    "skuId": "LAP-001",
6    "name": "Gaming Laptop X Pro",
7    "description": "Updated high performance gaming laptop",
8    "category": "Laptops",
9    "brand": "BrandX",
10   "price": 899900,
11   "itemCode": 123456789,
12   "length": 35,
13   "height": 2,
14   "width": 25,
15   "weight": 2000,
16   "dangerousLevel": 0
17 }

```

Response: 200 OK

```

1  {
2    "success": true,
3    "errorMessage": null,
4    "errorCode": null,
5    "status": 200,
6    "statusText": "OK",
7    "timestamp": "2025-12-07T10:30:02.260641Z",
8    "data": {
9      "skuId": "LAP-001",
10     "storeId": 10001,
11     "name": "Gaming Laptop X Pro",
12     "description": "Updated high performance gaming laptop",
13     "category": "Laptops",
14     "brand": "BrandX",
15     "price": 899900,
16     "itemCode": 123456789,
17     "isActive": true,
18     "length": 35,
19     "height": 2,
20     "width": 25,
21     "weight": 2000,
22     "dangerousLevel": 0,
23     "createdAt": "2025-11-01T10:00:00",
24     "updatedAt": "2025-12-07T10:30:02.260641"
25   }
26 }

```

Error Response: 404 Not Found (product not found)

```
1 {
2   "error": "Resource Not Found",
3   "message": "Product not found with id: LAP-999",
4   "timestamp": "2025-12-07T10:30:02.260641",
5   "status": 404
6 }
```

Error Response: 400 Bad Request (validation errors)

```
1 {
2   "error": "Validation Error",
3   "message": "Price is required",
4   "timestamp": "2025-12-07T10:30:02.260641",
5   "status": 400
6 }
```

3.8 DELETE /api/products/delete/{skuId}

Description: Delete a product by SKU ID

Path Parameters:

- skuId (required, String) - Unique product SKU identifier to delete

Example Request:

```
1 DELETE /api/products/delete/LAP-001
```

Response: 200 OK

```
1 {
2   "success": true,
3   "errorMessage": null,
4   "errorCode": null,
5   "status": 200,
6   "statusText": "OK",
7   "timestamp": "2025-12-07T10:30:02.260641Z",
8   "data": true
9 }
```

Error Response: 404 Not Found (product not found)

```
1 {
2   "error": "Resource Not Found",
3   "message": "Product not found with id: LAP-999",
4   "timestamp": "2025-12-07T10:30:02.260641",
5   "status": 404
6 }
```

Common Error Responses

500 Internal Server Error

```
1 {
2   "error": "Internal Server Error",
3   "message": "An unexpected error occurred. Please try again later.",
4   "timestamp": "2025-12-07T10:30:02.260641",
5   "status": 500
6 }
```

409 Conflict (Data Integrity Violation)

```
1 {
2   "error": "Data Integrity Violation",
3   "message": "A product with SKU ID 'LAP-001' already exists. Please use a different SKU ID.",
4   "timestamp": "2025-12-07T10:30:02.260641",
5   "status": 409
6 }
```

4. Cart APIs (protected via Gateway)

Base Path: /api/cart

Authentication: All endpoints require X-User-Id header (injected by API Gateway from JWT token)

4.1 GET /api/cart

Description: Get current user's cart (by userId from JWT via X-User-Id header).

Request:

- **Method:** GET
- **Path:** /api/cart
- **Headers:**
 - X-User-Id: {userId} (Long, required)

Response:

- **200 OK** - Cart retrieved successfully

Response Body Example:

```
1 {
2   "status": "SUCCESS",
3   "statusText": "OK",
4   "message": null,
5   "data": {
6     "userId": 12345,
7     "items": [
8       {
9         "skuId": "MTA-8935-56768",
10        "quantity": 2,
11        "price": 799900,
12        "addedAt": "2025-11-30T10:10:00Z",
13        "name": "Gaming Laptop X",

```

```

14     "description": "High performance gaming laptop with 16GB RAM,
15     1TB SSD...",
16     "category": "Laptops",
17     "brand": "BrandX",
18     "itemCode": 1001,
19     "length": 1000,
20     "height": 2332,
21     "width": 3232,
22     "weight": 234,
23     "dangerousLevel": 0
24   }
25 ],
26 "totalQuantity": 2,
27 "totalPrice": 1599800,
28 "updatedAt": "2025-11-30T10:15:00Z"
29 },
30 "timestamp": "2025-12-07T10:15:00Z"
31 }

```

Empty Cart Response:

```

1 {
2   "status": "SUCCESS",
3   "statusText": "OK",
4   "message": "Cart is empty",
5   "data": {
6     "userId": 12345,
7     "items": [],
8     "totalQuantity": 0,
9     "totalPrice": 0,
10    "updatedAt": "2025-11-30T10:15:00Z"
11  },
12  "timestamp": "2025-12-07T10:15:00Z"
13 }

```

Error Responses:

- **400 Bad Request** - Invalid userId format
- **500 Internal Server Error** - Server error

4.2 POST /api/cart/items

Description: Add an item to the current user's cart. If the item already exists, the quantity is replaced (not incremented).

Request:

- **Method:** POST
- **Path:** /api/cart/items
- **Headers:**
 - X-User-Id: {userId} (Long, required)
 - Content-Type: application/json
- **Request Body:**


```
1 {
2   "skuId": "MTA-8935-56768",
3   "quantity": 2
4 }
```

Request Body Fields:

Field	Type	Required	Description
skuId	String	Yes	Product SKU identifier
quantity	Integer	Yes	Quantity to add (must be > 0)

Response:

- **201 Created** - New item added to cart
- **200 OK** - Existing item updated in cart
- **200 OK** - Item removed (if product no longer exists in product service)
- **200 OK** - Cart is empty (if all items were removed)

Response Body (201 Created - New Item):

```
1 {
2   "status": "SUCCESS",
3   "statusText": "Created",
4   "message": null,
5   "data": true,
6   "timestamp": "2025-12-07T10:15:00Z"
7 }
```

Response Body (200 OK - Updated Item):

```
1 {
2   "status": "SUCCESS",
3   "statusText": "Updated",
4   "message": null,
5   "data": true,
6   "timestamp": "2025-12-07T10:15:00Z"
7 }
```

Response Body (200 OK - Cart Empty):

```
1 {
2   "status": "SUCCESS",
3   "statusText": "OK",
4   "message": "Cart is empty",
5   "data": {
6     "userId": 12345,
7     "items": [],
8     "totalQuantity": 0,
9     "totalPrice": 0,
```

```
10     "updatedAt": "2025-11-30T10:15:00Z"
11   },
12   "timestamp": "2025-12-07T10:15:00Z"
13 }
```

Error Responses:

- **400 Bad Request** - Invalid request body (missing skuId or quantity, invalid quantity)
 - **404 Not Found** - Product with given skuId does not exist (only for new items)
 - **500 Internal Server Error** - Server error or product service unavailable
-

4.3 DELETE /api/cart/items/{itemId}

Description: Remove an item from the cart by item SKU ID.

Request:

- **Method:** DELETE
- **Path:** /api/cart/items/{itemId}
- **Path Parameters:**
 - **itemId** : String (required) - SKU ID of the item to remove
- **Headers:**
 - **X-User-Id** : {userId} (Long, required)

Example Request:

```
1 DELETE /api/cart/items/MTA-8935-56768
2 Headers:
3   X-User-Id: 12345
```

Response:

- **200 OK** - Item removed successfully

Response Body:

```
1 {
2   "status": "SUCCESS",
3   "statusText": "OK",
4   "message": "Successfully removed an item from the cart",
5   "data": true,
6   "timestamp": "2025-12-07T10:15:00Z"
7 }
```

Error Responses:

- **404 Not Found** - Cart not found for userId or item not found in cart
- **400 Bad Request** - Invalid userId format
- **500 Internal Server Error** - Server error

4.4 PATCH /api/cart/items/{itemSku}

Description: Increase or decrease the quantity of an item in the cart by a specified amount.

Request:

- **Method:** PATCH
- **Path:** /api/cart/items/{itemSku}
- **Path Parameters:**
 - `itemSku` : String (required) - SKU ID of the item to update
- **Query Parameters:**
 - `quantity` : Integer (required) - Amount to increase or decrease
 - `operation` : String (required) - Either "increase" or "decrease" (case-insensitive)
- **Headers:**
 - `X-User-Id` : {userId} (Long, required)

Example Request:

```
1 PATCH /api/cart/items/MTA-8935-56768?quantity=2&operation=increase
2 Headers:
3   X-User-Id: 12345
```

Query Parameters:

Parameter	Type	Required	Description
<code>quantity</code>	Integer	Yes	Amount to increase/decrease (must be > 0)
<code>operation</code>	String	Yes	"increase" or "decrease" (case-insensitive)

Response:

- **200 OK** - Quantity updated successfully
- **200 OK** - Item removed (if quantity decreased to 0 or below)

Response Body (200 OK - Updated):

```
1 {
2   "status": "SUCCESS",
```

```

3  "statusText": "Updated",
4  "message": "Successfully updated item quantity in the cart",
5  "data": true,
6  "timestamp": "2025-12-07T10:15:00Z"
7  }

```

Response Body (200 OK - Removed):

```

1  {
2    "status": "SUCCESS",
3    "statusText": "Removed",
4    "message": "Item quantity decreased to 0 or below. Item removed from
    cart.",
5    "data": true,
6    "timestamp": "2025-12-07T10:15:00Z"
7  }

```

Error Responses:

- **400 Bad Request** - Invalid parameters (missing quantity/operation, invalid operation value, invalid quantity)
- **404 Not Found** - Cart not found for userId or item not found in cart
- **500 Internal Server Error** - Server error

Common Response Structure

All successful responses follow this structure:

```

1  {
2    "status": "SUCCESS",
3    "statusText": "OK" | "Created" | "Updated" | "Removed",
4    "message": "string | null",
5    "data": <response_data>,
6    "timestamp": "ISO-8601 timestamp"
7  }

```

Common Error Response Structure

```

1  {
2    "error": "Error Type",
3    "message": "Detailed error message",
4    "timestamp": "ISO-8601 timestamp",
5    "status": <http_status_code>
6  }Response Wrapper Structure

```

All successful responses are wrapped in `ApiResponse<T>` format:

```

1  {
2    "success": boolean,           // true for success, false for errors
3    "errorMessage": string|null, // Error message if failed
4    "errorCode": string|null,    // Error code if failed
5    "status": number,           // HTTP status code
6    "statusText": string,       // HTTP status text (e.g., "OK", "CREAT
    ED")
7    "timestamp": string,        // ISO 8601 timestamp
8    "data": object|null        // Response payload (null if error)

```

4. Search APIs (public)

1. Full-Text Search

Endpoint: GET /api/search

Description: Full-text search across product name and description fields.

Response:

- **Status Code:** 200 OK
- **Content-Type:** application/json

```
1 {
2   "status": "success",
3   "message": "Search completed successfully",
4   "data": {
5     "content": [
6       {
7         "skuId": "MTA-4537-43598",
8         "storeId": 1,
9         "name": "milk",
10        "description": "freshwholemilk",
11        "category": "drinks",
12        "brand": "nestle",
13        "price": 5000,
14        "itemCode": 1234567890,
15        "isActive": true,
16        "length": 10,
17        "height": 5,
18        "width": 8,
19        "weight": 250,
20        "dangerousLevel": 0,
21        "createdAt": "2025-12-04",
22        "updatedAt": "2025-12-04"
23      }
24    ],
25    "pageable": {
26      "pageNumber": 0,
27      "pageSize": 20,
28      "sort": {
29        "sorted": false
30      }
31    },
32    "totalElements": 1,
33    "totalPages": 1,
34    "last": true,
35    "first": true,
36    "numberOfElements": 1,
37    "size": 20,
38    "number": 0,
39    "empty": false
40  },
41  "timestamp": "2025-12-05T10:30:00"
42 }
```

Error Responses:

Status Code	Description	Example Response
400 Bad Request	Empty query parameter	<pre> {"status": "error", "message": "Search query cannot be empty", "data": null, "timestamp": "2025-12- 05T10:30:00"} </pre>
500 Internal Server Error	Server error	<pre> {"status": "error", "message": "Error performing search: [error details]", "data": null, "timestamp": "2025-12- 05T10:30:00"} </pre>